

Atelier IFORD × GDSG · Yaoundé, 28 juillet 2026

AUTEUR-RICE
Ramesesse Dzita (GDSG)

DATE DE PUBLICATION
28 juillet 2026

1 Objectifs et public cible

Ce deuxième jour approfondit **sf** au-delà des premiers contacts de J1. On apprend à **lire et écrire** des fichiers spatiaux dans tous les formats courants, à **reprojeter de façon disciplinée**, et à **opérer géométriquement** sur les couches : buffer, intersection, union, dissolve, Voronoï. Ce sont les briques qu'on combinera tout le reste de la formation.

À la fin du jour, chaque participant doit pouvoir :

1. Lire un Shapefile, un GeoJSON, un GeoPackage avec une seule fonction.
2. Reprojecter entre WGS 84, UTM 32 N / 33 N, et Lambert Cameroun selon la question posée.
3. Construire des buffers, intersections, unions, dissolves dans un pipeline **dplyr**.
4. Détecter et réparer des géométries malformées après reprojection.
5. Produire un diagramme de Voronoï à partir des chefs-lieux régionaux.

Les exercices Q1 à Q6 sont dans **exercice.qmd**. Le corrigé est dans **corrige.qmd**.

2 Pré-requis

On charge les packages nécessaires et le helper **fetch_data.R** qui résout les chemins vers les données GADM. Le bloc suivant doit tourner sans erreur — sinon retour à **environnement_technique/install_packages.R**.

```
library(sf)
library(dplyr)
library(tibble)
library(ggplot2)

# Chemin relatif depuis le .qmd (pedagogie/JXX_.../) car here::here()
# peut etre dupe par pedagogie/_quarto.yml et nous proposer
# pedagogie/pedagogie/_commons/... inexistant.
source("../_commons/helpers/fetch_data.R")
```

3 Première session — Lecture et écriture multi-format

 **~1 heure**. C'est la session technique de la matinée — on doit savoir lire **n'importe quel** fichier qu'un participant nous tend.

3.1 Lire un GeoJSON

GADM v4.1 publie ses limites administratives au format GeoJSON. `read_sf()` détecte automatiquement le format à partir de l'extension. Le helper `fetch_gadm_cameroon(level)` retourne le chemin local du fichier (ou s'arrête avec un message clair s'il manque). Le résultat est un objet `sf` prêt à manipuler.

```
adm1 <- read_sf(fetch_gadm_cameroon(1))
class(adm1)
```

```
[1] "sf"          "tbl_df"      "tbl"         "data.frame"
```

```
nrow(adm1)
```

```
[1] 10
```

```
names(adm1)
```

```
[1] "GID_1"      "GID_0"      "COUNTRY"    "NAME_1"     "VARNAME_1"  "NL_NAME_1"
[7] "TYPE_1"     "ENGTYPE_1"  "CC_1"       "HASC_1"     "ISO_1"      "geometry"
```

3.2 Inspecter sans tout charger

Sur un fichier inconnu, le réflexe est de **lister les couches** avant de lire. `st_layers()` te dit ce qu'il y a dedans sans charger les géométries — utile pour un GeoPackage multi-couches ou un gros Shapefile.

```
st_layers(fetch_gadm_cameroon(2))
```

name	geomtype	driver	features	fields	crs
<chr>	<list>	<chr>	<dbl>	<dbl>	<list>
1 gadm41_CMR_2	<chr [1]>	GeoJSON	58	13	<S3: crs>
1 row					

3.3 Filtrer dès la lecture (SQL embarqué)

Sur un gros fichier (ADM3 = 365 arrondissements), on peut **ne charger qu'une portion** via la clause `query =`. GDAL exécute ce SQL avant de retourner l'objet R. Pour les fichiers > 100 Mo c'est la différence entre 1 seconde et 30 secondes.

```
litoral <- read_sf(
  fetch_gadm_cameroon(3),
  query = "SELECT * FROM gadm41_CMR_3 WHERE NAME_1 = 'Littoral'"
)
nrow(litoral)
```

```
[1] 35
```

```
head(litoral$NAME_2, 5)
```

```
[1] "Moungo" "Moungo" "Moungo" "Moungo" "Moungo"
```

3.4 Écrire en GeoPackage

`write_sf()` est le symétrique de `read_sf()`. On écrit en GeoPackage parce que c'est le format moderne par défaut. L'argument `delete_dsn = TRUE` écrase un fichier existant ; sans lui, `sf` refuse d'écraser pour ta sécurité.

```
out_path <- here("pedagogie", "J02_sf_CRS_vecteurs", "outputs", "CMR_litoral.gpkg")
dir.create(dirname(out_path), showWarnings = FALSE, recursive = TRUE)

write_sf(litoral, out_path,
         layer = "litoral_adm3",
         delete_dsn = TRUE)
```

4 Deuxième session — CRS appliqué

 **~1 heure.** On rappelle la théorie de J1 par la pratique : trois projections, trois usages.

4.1 Comparer trois CRS sur le Cameroun

On va reprojeter les 10 régions ADM1 dans trois CRS différents (Lambert Cameroun, UTM 32 N, Web Mercator), calculer la superficie totale dans chaque système, et comparer au chiffre officiel (475 442 km²). Cela démontre concrètement ce qu'on a vu en théorie.

```
adm1_lambert <- st_transform(adm1, 3119) # Lambert Cameroun
adm1_utm32   <- st_transform(adm1, 32632) # UTM zone 32 N
adm1_3857    <- st_transform(adm1, 3857) # Web Mercator

total_km2 <- function(x) sum(as.numeric(st_area(x))) / 1e6

tibble(
  CRS      = c("Lambert CMR (3119)", "UTM 32 N (32632)", "Web Mercator (3857)"),
  superficie_km2 = c(total_km2(adm1_lambert),
                    total_km2(adm1_utm32),
                    total_km2(adm1_3857)) |> round(),
  ecart_pct = round((c(total_km2(adm1_lambert),
                    total_km2(adm1_utm32),
                    total_km2(adm1_3857)) - 475442) / 475442 * 100, 2)
)
```

CRS	superficie_km2	ecart_pct
<chr>	<dbl>	<dbl>
Lambert CMR (3119)	466238	-1.94
UTM 32 N (32632)	468069	-1.55
Web Mercator (3857)	474646	-0.17

3 rows

Le tableau confirme la théorie : Lambert Cameroun est le plus proche du chiffre officiel (faible déformation sur l'ensemble du pays), UTM 32 N est précis à l'ouest mais sous-estime à l'est, Web Mercator surestime de plusieurs pourcents.

4.2 La règle d'or appliquée : reprojecter le plus tard possible

On veut sélectionner les régions du nord (latitude > 7°). C'est un filtre **attributaire**, donc on peut le faire **avant** toute reprojection — en WGS 84 directement. Une fois la sélection faite, on reprojette **uniquement** pour calculer la superficie.

```
# Filtrer en WGS 84 (les coordonnees sont en degres)
regions_nord <- adm1 |>
  filter(NAME_1 %in% c("Nord", "Adamaoua", "Extrême-Nord", "Est"))

# Reprojecter SEULEMENT maintenant pour le calcul de surface
regions_nord_utm <- st_transform(regions_nord, 32632)
regions_nord_utm$superficie_km2 <- as.numeric(st_area(regions_nord_utm)) / 1e6

regions_nord_utm |>
  st_drop_geometry() |>
  select(NAME_1, superficie_km2) |>
  arrange(desc(superficie_km2))
```

NAME_1	superficie_km2
<chr>	<dbl>
Est	109782.84
Nord	66802.86
Adamaoua	64266.74
Extrême-Nord	34580.85

4 rows

5 Troisième session — Opérations vectorielles

🕒 ~1h30. Le cœur du jour. On enchaîne `st_buffer`, `st_intersection`, `st_union`, `st_centroid`, `st_voronoi` en pipeline.

5.1 Buffer : zone tampon autour de Yaoundé

`st_buffer(x, dist = 50000)` retourne tous les points à moins de 50 km de `x`. La distance est dans l'**unité du CRS** — il faut donc reprojeter en mètres (UTM 32 N) avant d'appeler `st_buffer` avec une distance en mètres.

```
yaounde_pt <- st_sfc(st_point(c(11.5021, 3.8480)), crs = 4326)
yaounde_utm <- st_transform(yaounde_pt, 32632)

buffer_50km <- st_buffer(yaounde_utm, dist = 50000) # 50 km en metres

# Verifier le rayon
sqrt(as.numeric(st_area(buffer_50km)) / pi) / 1000 # devrait afficher ~50 km
```

```
[1] 49.98858
```

5.2 Intersection : quels arrondissements tombent dans ce buffer ?

`st_intersection(x, y)` retourne la portion de chaque feature de `y` qui tombe dans `x`. C'est exactement la question « quels arrondissements (ADM3) sont dans le buffer de 50 km autour de Yaoundé ? ».

```
adm3 <- read_sf(fetch_gadm_cameroon(3))
adm3_utm <- st_transform(adm3, 32632)

dans_buffer <- st_intersection(buffer_50km, adm3_utm)
nrow(dans_buffer)
```

```
NULL
```

```
head(dans_buffer$NAME_3, 10)
```

```
NULL
```

5.3 Union : silhouette du Cameroun

`st_union(adm1)` fusionne tous les polygones en un seul. On obtient la **silhouette nationale**, utile pour les fonds de carte ou les frontières dans `ggplot`.

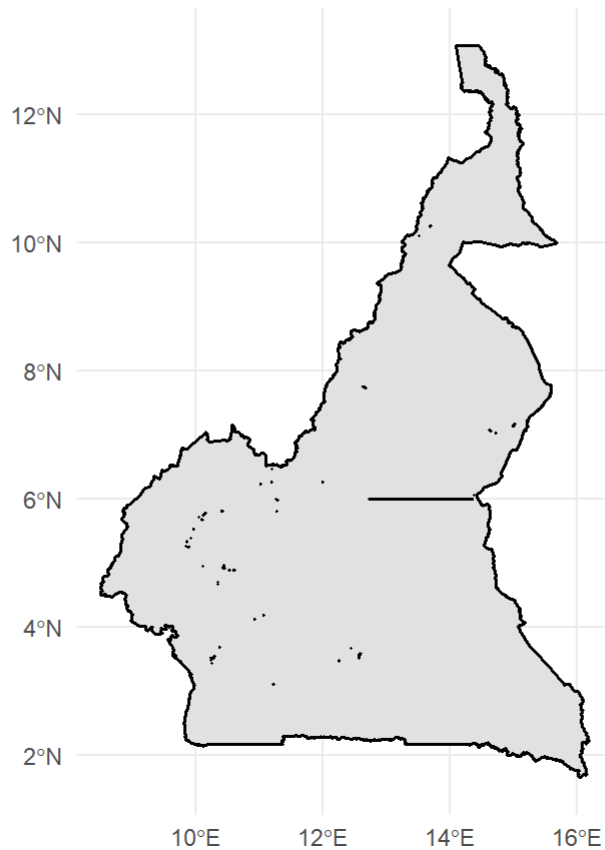
```
cmr_silhouette <- st_union(adm1)
class(cmr_silhouette)
```

```
[1] "sfc_POLYGON" "sfc"
```

```
ggplot() +
  geom_sf(data = cmr_silhouette, fill = "grey90", color = "black", linewidth = 0.6) +
  labs(title = "Silhouette du Cameroun",
       subtitle = "st_union(adm1) - 10 polygones fusionnés en 1") +
  theme_minimal()
```

Silhouette du Cameroun

st_union(adm1) — 10 polygones fusionnés en 1



5.4 Dissolve : reconstituer les régions depuis les arrondissements

Le pattern **dissolve** est central en SIG : agréger une couche fine par identifiant d'un niveau supérieur. Ici on reconstruit les 10 régions (ADM1) depuis les 365 arrondissements (ADM3). En `sf` + `dplyr`, ça se lit comme une phrase : « grouper par `NAME_1`, puis résumer en fusionnant les géométries ».

```
adm1_from_adm3 <- adm3 |>
  group_by(NAME_1) |>
  summarise(n_arrondissements = n(),
            geometry = st_union(geometry),
            .groups = "drop")

nrow(adm1_from_adm3) # 10 regions
```

```
[1] 10
```

```
sum(adm1_from_adm3$n_arrondissements) # 365 arrondissements
```

```
[1] 360
```

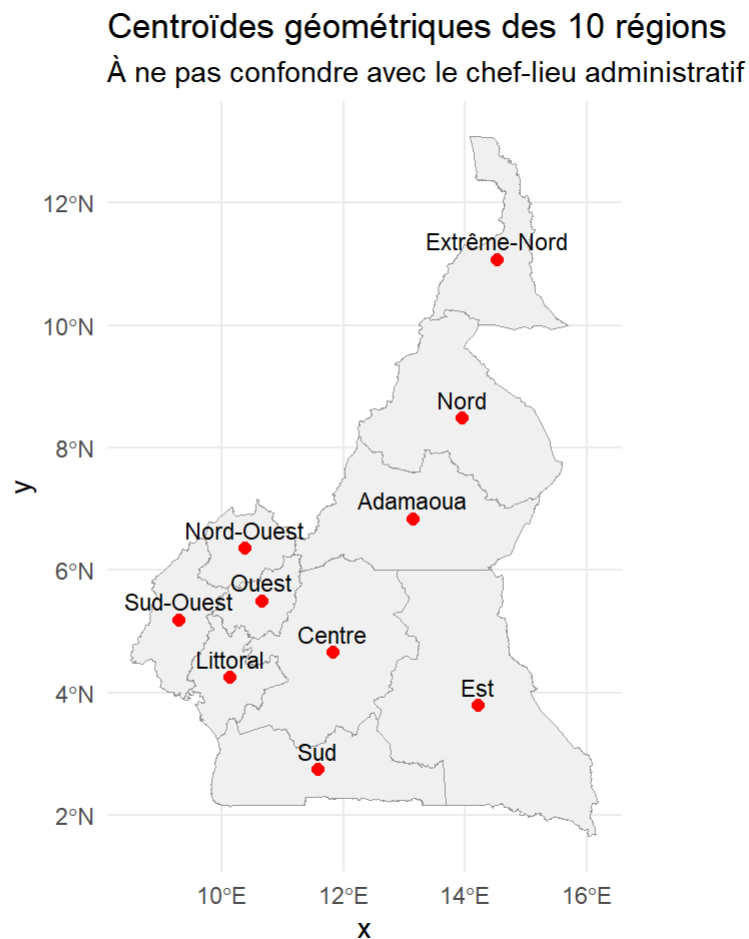
5.5 Centroïdes : un point par région

`st_centroid()` réduit chaque polygone à son centre géométrique. Utile pour placer une étiquette, mais à ne **pas** confondre avec le chef-lieu réel.

```
centroïdes_adm1 <- st_centroid(adm1)
class(centroïdes_adm1)
```

```
[1] "sf"          "tbl_df"      "tbl"        "data.frame"
```

```
ggplot() +
  geom_sf(data = adm1, fill = "grey95", color = "grey60") +
  geom_sf(data = centroïdes_adm1, color = "red", size = 2) +
  geom_sf_text(data = centroïdes_adm1, aes(label = NAME_1),
              nudge_y = 0.3, size = 3) +
  labs(title = "Centroïdes géométriques des 10 régions",
       subtitle = "À ne pas confondre avec le chef-lieu administratif") +
  theme_minimal()
```



5.6 Voronoï des chefs-lieux

On construit un tibble des **vrais** chefs-lieux régionaux (coordonnées GPS approximatives), on en fait un objet `sf`, on calcule le diagramme de Voronoï, et on intersecte avec la silhouette nationale pour limiter au territoire camerounais.

```

chefs_lieux <- tibble(
  region = c("Adamaoua", "Centre", "Est", "Extrême-Nord", "Littoral",
    "Nord", "Nord-Ouest", "Ouest", "Sud", "Sud-Ouest"),
  ville = c("Ngaoundéré", "Yaoundé", "Bertoua", "Maroua", "Douala",
    "Garoua", "Bamenda", "Bafoussam", "Ebolowa", "Buea"),
  lon = c(13.5870, 11.5021, 13.6818, 14.3158, 9.7679,
    13.3917, 10.1591, 10.4170, 11.1500, 9.2402),
  lat = c( 7.3214,  3.8480,  4.5775, 10.5910, 4.0511,
    9.3019,  5.9597,  5.4778,  2.9000, 4.1546)
) |>
  st_as_sf(coords = c("lon", "lat"), crs = 4326)

# Construire le Voronoi (st_voronoi attend un MULTIPOINT)
chefs_pts_union <- st_union(chefs_lieux)
voronoi_box <- st_voronoi(chefs_pts_union)

# Decouper en cellules individuelles et associer chaque cellule a son chef-lieu
voronoi_cells <- st_collection_extract(voronoi_box, "POLYGON") |>
  st_sf(crs = 4326)

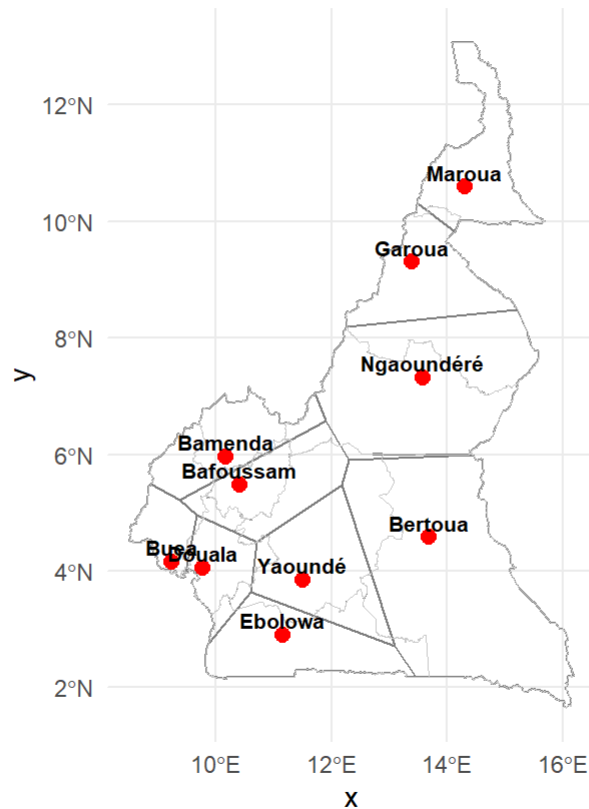
# Intersecter avec la silhouette nationale pour rester dans le Cameroun
voronoi_cmr <- st_intersection(voronoi_cells, st_union(adm1))

ggplot() +
  geom_sf(data = voronoi_cmr, fill = NA, color = "grey50", linewidth = 0.4) +
  geom_sf(data = adm1, fill = NA, color = "grey80", linewidth = 0.2) +
  geom_sf(data = chefs_lieux, color = "red", size = 2.5) +
  geom_sf_text(data = chefs_lieux, aes(label = ville),
    nudge_y = 0.25, size = 2.8, fontface = "bold") +
  labs(title = "Zones d'influence des chefs-lieux régionaux",
    subtitle = "Diagramme de Voronoï coupé sur la silhouette du Cameroun",
    caption = "Source : GADM v4.1 · IFORD × GDSG 2026") +
  theme_minimal()

```


Zones d'influence des chefs-lieux régionaux

Diagramme de Voronoï coupé sur la silhouette du Cameroun



Source : GADM v4.1 · IFORD × GDSG 2026

6 Quatrième session — Validation des géométries

🕒 ~30 minutes. Court mais critique : sans géométries valides, toutes les opérations suivantes plantent.

6.1 Diagnostic : `st_is_valid()`

`st_is_valid()` retourne `TRUE` / `FALSE` pour chaque feature. C'est le réflexe **après toute reprojection** : vérifier qu'aucune géométrie n'a été pincée.

```
adm3_lambert <- st_transform(adm3, 3119)
table(st_is_valid(adm3_lambert))
```

TRUE
360

6.2 Réparation : `st_make_valid()`

Si une géométrie est invalide, `st_make_valid()` la répare en utilisant l'algorithme GEOS. L'opération est silencieuse (pas de message en sortie) mais l'objet retourné a toutes ses géométries valides.

```
adm3_lambert_clean <- st_make_valid(adm3_lambert)
all(st_is_valid(adm3_lambert_clean))
```

[1] TRUE

7 Cinquième session — Exporter la chaîne complète

 ~30 minutes.

7.1 GeoPackage multi-couches

Le format `.gpkg` peut contenir **plusieurs couches** dans un seul fichier. On y stocke ADM1 dissous, ADM2, ADM3, et nos cellules Voronoï — un seul fichier à partager au prochain participant.

```
out_path <- here("pedagogie", "J02_sf_CRS_vecteurs", "outputs",
                 "CMR_admin_lambert.gpkg")
dir.create(dirname(out_path), showWarnings = FALSE, recursive = TRUE)

write_sf(st_transform(adm1, 3119), out_path, layer = "adm1", delete_dsn = TRUE)
write_sf(st_transform(adm3, 3119), out_path, layer = "adm3", delete_layer = TRUE)
write_sf(st_transform(voronoi_cmr, 3119), out_path,
         layer = "voronoi_chefs_lieux", delete_layer = TRUE)

st_layers(out_path)
```

8 Sixième session — Synthèse et devoir

 ~30 minutes.

8.1 Ce qu'on a vu

- Une seule fonction pour lire / écrire tous les formats (`read_sf` / `write_sf`).
- GeoPackage est le format moderne par défaut, plusieurs couches dans un fichier.
- Filtrer dès la lecture avec `query =` pour les gros fichiers.
- Trois CRS du Cameroun : 4326 (échange), UTM 32 N / 33 N (régional), Lambert 3119 (national).
- La règle d'or : reprojeter le plus tard possible.
- Six opérations vectorielles : `buffer`, `intersection`, `union`, `centroid`, `voronoi`, `make_valid`.
- Pattern dissolve : `group_by + summarise(geometry = st_union(geometry))`.

8.2 Pour demain (J3)

Données raster avec terra : concepts raster (résolution, étendue, CRS, bandes), sept opérations fondamentales (`crop`, `mask`, `resample`, `aggregate`, `project`, `extract`, `global`), algèbre raster, extraction

zonale par polygones administratifs. Cas pratique : MNT SRTM 30 m du Cameroun et élévation moyenne par région.

8.3 Devoir individuel (Q6 — 30 min)

Énoncé dans [exercice.qmd](#) : produire un GeoPackage avec deux couches — la couche départementale (ADM2) avec superficie et nombre d'arrondissements, et la couche régionale (ADM1) reconstituée par dissolve. Corrigé dans [corrige.qmd](#).

9 Sources et licences

Donnée	Source	Licence
GADM v4.1 ADM1, ADM2, ADM3 Cameroun	https://gadm.org/download_country.html	Libre académique
Chefs-lieux régionaux (coords)	OpenStreetMap + connaissance terrain	ODbL

Code distribué sous **CC-BY 4.0**.

9.1 Références méthodologiques

Pebesma ([2018](#)) ; Bureau Central des Recensements et des Études de Population (BUCREP) ([2026](#)).

Les références

Bureau Central des Recensements et des Études de Population (BUCREP). 2026. *Quatrième Recensement Général de la Population et de l'Habitat du Cameroun (RGPH 4)*. République du Cameroun.

Pebesma, Edzer. 2018. « Simple Features for R: Standardized Support for Spatial Vector Data ». *The R Journal* 10 (1): 439-46.